

CSCI-3171 Network Computing

Lab 5 – Winter 2026

Submission Instructions:

Submit **one PDF file** for your lab report on Brightspace under the Lab 5 portal. Include the following:

1. **Summary Report:** Provide a summary of your observations for the given questions.
2. **Graphs and Numerical Values:** Include any relevant graphs and numerical values derived from your experiments.
3. **Screenshots:** If necessary, include screenshots. However, note that command responses can be easily copy-pasted into the report without the need for heavy screenshots.
4. **Identification:** Ensure your full name and banner ID are clearly mentioned in the report.

Late submission: Refer to the syllabus on Brightspace for a complete description of the late submission policy.

Table of Contents

1. Introduction.....	1
2. nslookup	1
3. The DNS cache on your computer.....	3
4. Tracing DNS with Wireshark.....	4

1. Introduction

The Domain Name System (DNS) translates hostnames to IP addresses, fulfilling a critical role in the Internet infrastructure. In this lab, we'll take a closer look at the client side of DNS. Recall that the client's role in the DNS is relatively simple – a client sends a *query* to its local DNS server, and receives a *response* back. However, much can go on “under the covers,” invisible to a DNS client, as the hierarchical DNS servers communicate with each other to either recursively or iteratively resolve the client's DNS query. From the DNS client's standpoint, however, the protocol is quite simple – a query is formulated to the local DNS server and a response is received from that server.

Before beginning this lab, you'll probably want to review DNS concepts. In particular, you may want to review the material on **local DNS servers**, **DNS caching**, **DNS records and messages**, and the **TYPE field** in the DNS record.

2. nslookup

Let's start our investigation of the DNS by examining the nslookup command, which will invoke the underlying DNS services to implement its functionality. The nslookup command is available in most Microsoft, Apple IOS, and Linux operating systems. To run nslookup you just type the nslookup command on the command line in a DOS window, Mac terminal window, or Linux shell.

In its most basic operation, nslookup allows the host running nslookup to query any specified DNS server for a DNS record. The queried DNS server can be a root DNS server, a top-level-domain (TLD) DNS server, an authoritative DNS server, or an intermediate DNS server. For example, nslookup can be used to retrieve

a “Type=A” DNS record that maps a hostname (e.g., `www.lahoud.fr`) to its IP address. To accomplish this task, `nslookup` sends a DNS query to the specified DNS server (or the default local DNS server for the host on which `nslookup` is run, if no specific DNS server is specified), receives a DNS response from that DNS server, and displays the result.

Let’s take `nslookup` out for a spin! We’ll first run `nslookup` on the Linux command line on the `timberlea.cs.dal.ca` host located in the Faculty of Computer Science at Dalhousie University, where the local name server is named `ns1.cs.dal.ca` (which has an IP address `129.173.22.37`). Let’s try `nslookup` in its simplest form:

```
lahoud@timberlea:~$ nslookup www.lahoud.fr
Server:          129.173.22.37
Address:         129.173.22.37#53
```

```
Non-authoritative answer:
www.lahoud.fr canonical name = lahoud.fr.
Name:   lahoud.fr
Address: 213.186.33.24
```

In this example the `nslookup` command is given one argument, a hostname (`www.lahoud.fr`). In words, this command is saying “please send me the IP address for the host `www.lahoud.fr`” As shown in the screenshot, the response from this command provides two pieces of information: (1) the name and IP address of the DNS server that provides the answer – in this case the local DNS server at FCS; and (2) the answer itself, which is the canonical host name and IP address of `www.lahoud.fr`.

Although the response came from the local DNS server (with IP address `129.173.22.37`) at FCS, it is quite possible that this local DNS server iteratively contacted several other DNS servers to get the answer.

In addition to using `nslookup` to query for a DNS “Type=A” record, we can also use `nslookup` to query for a “TYPE=NS” record, which returns the hostname (and its IP address) of an authoritative DNS server that knows how to obtain the IP addresses for hosts in the authoritative server’s domain.

```
lahoud@timberlea:~$ nslookup -type=NS concordia.ca
Server:          129.173.22.37
Address:         129.173.22.37#53
```

```
Non-authoritative answer:
concordia.ca nameserver = ns-b.concordia.ca.
concordia.ca nameserver = ns-a.concordia.ca.
concordia.ca nameserver = ns2.zonerisq.ca.
concordia.ca nameserver = ns1.cc.umanitoba.ca.
concordia.ca nameserver = ns1.zonerisq.ca.
```

```
Authoritative answers can be found from:
ns-a.concordia.ca    internet address = 132.205.152.111
```

In the previous example, we’ve invoked `nslookup` with the option “-type=NS” and the domain “concordia.ca”. This causes `nslookup` to send a query for a type-NS record to the default local DNS server. In words, the query is saying, “please send me the host names of the authoritative DNS for `concordia.ca`”. (When the `-type` option is not used, *nslookup* uses the default, which is to query for type A records.) The

answer, displayed in the above screenshot, first indicates the DNS server that is providing the answer (which is the default local Dal FCS DNS server) along with five Concordia DNS name servers. Each of these servers is indeed an authoritative DNS server for the hosts on the Concordia campus. However, `nslookup` also indicates that the answer is “non-authoritative,” meaning that this answer came from the cache of some server rather than from an authoritative Concordia DNS server. Finally, the answer also includes the IP addresses of the authoritative DNS servers at Concordia. (Even though the type-NS query generated by `nslookup` did not explicitly ask for the IP addresses, the local DNS server returned these “for free” and `nslookup` displays the result.)

`nslookup` has a number of additional options beyond “-type=NS” that you might want to explore. Here’s a site with screenshots of ten popular `nslookup` uses: <https://www.cloudns.net/blog/10-most-used-nslookup-commands/> and here are the “man pages” for `nslookup`: <https://linux.die.net/man/1/nslookup>.

Lastly, we sometimes might be interested in discovering the name of the host associated with a given IP address, i.e., the reverse of the lookup in the first example (where the host’s name was known/specified and the host’s IP address was returned). `nslookup` can also be used to perform this so-called “reverse DNS lookup.” In the following example, we specify an IP address as the `nslookup` argument (205.189.32.200 in this example) and `nslookup` returns the host name with that address (`mtrl2rtr1.network.canarie.ca` in this example)

```
lahoud@timberlea:~$ nslookup 205.189.32.200
200.32.189.205.in-addr.arpa      name = mtrl2rtr1.network.canarie.ca.
```

Now that we’ve provided an overview of `nslookup`, it’s time for you to test drive it yourself. Follow these steps on your computer or on the CS server `timberlea.cs.dal.ca`. Record your findings using screenshots or extracts from command responses, as outlined in the submission guidelines.

- Q1.** Run `nslookup` to obtain the IP address of the web server for the KTH Royal Institute of Technology in Stockholm, Sweden: `www.kth.se`. What is the IP address of `www.kth.se`?
- Q2.** What is the IP address of the DNS server that provided the answer to your `nslookup` command in question 1 above?
- Q3.** Did the answer to your `nslookup` command in question 1 above come from an authoritative or non-authoritative server?
- Q4.** Use the `nslookup` command to determine the name of the authoritative name server for the `kth.se` domain. What is that name? (If there are more than one authoritative servers, what is the name of the first authoritative server returned by `nslookup`)? If you had to find the IP address of that authoritative name server, how would you do so?

3. The DNS cache on your computer

From the description of iterative and recursive DNS query resolution, you might think that the local DNS server must be contacted *every* time an application needs to translate from a hostname to an IP address. That’s not always true in practice!

Most hosts (e.g., your personal computer) keep a *cache* of recently retrieved DNS records (sometimes called a *DNS resolver cache*), just like many Web browsers keep a cache of objects recently retrieved by HTTP. When DNS services need to be invoked by a host, that host will first check if the DNS record needed

is resident in this host's DNS cache; if the record is found, the host will not even bother to contact the local DNS server and will instead use this cached DNS record. A DNS record in a resolver cache will eventually timeout and be removed from the resolver cache, just as records cached in a local DNS server will timeout.

You can also explicitly clear the records in your DNS cache. There's no harm in doing so – it will just mean that your computer will need to invoke the distributed DNS service next time it needs to use the DNS name resolution service, since it will find no records in the cache. On a Mac computer, you can enter the following command into a terminal window to clear your DNS resolver cache:

```
sudo killall -HUP mDNSResponder
```

On Windows computer you can enter the following command at the command prompt:

```
ipconfig /flushdns
```

and on a Linux computer, enter:

```
sudo systemd-resolve --flush-caches
```

and for recent Ubuntu:

```
sudo resolvectl flush-caches
```

4. Tracing DNS with Wireshark

Now that we are familiar with nslookup and clearing the DNS resolver cache, we're ready to get down to some serious business. Let's first capture the DNS messages that are generated by ordinary Web-surfing activity.

- Clear the DNS cache in your host, as described above.
- Open your Web browser in private/incognito mode.
- Open Wireshark and enter `ip.addr == <your_IP_address>` into the display filter, where `<your_IP_address>` is the IPv4 address of your computer¹. With this filter, Wireshark will only display packets that either originate from, or are destined to, your host.
- Start packet capture in Wireshark.
- With your browser, visit the Web page: <http://www.lahoud.fr/lab3.html>
- Stop packet capture.

Answer the following questions. Use screenshots and highlight/circle the information to indicate where in the packet you have found the information that answers a question.

¹ If you're not sure how to find the IP address of your computer, you can search the Web for articles for your operating system. Windows 10 info is [here](#); Mac info is [here](#); Linux info is [here](#)

- Q5.** Locate the first DNS query message resolving the name `www.lahoud.fr`. What is the packet number² in the trace for the DNS query message? Is this query message sent over UDP or TCP?
- Q6.** Now locate the corresponding DNS response to the initial DNS query. What is the packet number in the trace for the DNS response message? Is this response message received via UDP or TCP?
- Q7.** What is the destination port for the DNS query message? What is the source port of the DNS response message?
- Q8.** To what IP address is the DNS query message sent?
- Q9.** Examine the DNS query message. How many “questions” does this DNS message contain? How many “answers” does it contain?
- Q10.** Examine the DNS response message to the initial query message. How many “questions” does this DNS message contain? How many “answers” does it contain?
- Q11.** The web page for the base file `http://www.lahoud.fr/lab3.html` references the image object `http://www.lahoud.fr/network-comics.jpg`, which, like the base webpage, is on `lahoud.fr`. What is the packet number in the trace for the initial HTTP GET request for the base file `http://www.lahoud.fr`? What is the packet number in the trace of the DNS query made to resolve `www.lahoud.fr` so that this initial HTTP request can be sent to the `www.lahoud.fr` IP address? What is the packet number in the trace of the received DNS response? What is the packet number in the trace for the HTTP GET request for the image object `http://www.lahoud.fr/network-comics.jpg`? What is the packet number in the DNS query made to resolve `www.lahoud.fr` so that this second HTTP request can be sent to the `www.lahoud.fr` IP address? Discuss how DNS caching affects the answer to this last question.

Now let's play with `nslookup`:

- Start packet capture.
- Do an `nslookup` on `www.lahoud.fr`
- Stop packet capture.

Let's look at the first type A query.

- Q12.** What is the destination port for the DNS query message? What is the source port of the DNS response message?
- Q13.** To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server?
- Q14.** Examine the DNS query message. What “Type” of DNS query is it? Does the query message contain any “answers”?
- Q15.** Examine the DNS response message to the query message. How many “questions” does this DNS response message contain? How many “answers”?

Last, let's use `nslookup` to issue a command that will return a type NS DNS record, Enter the following command:

```
nslookup -type=NS lahoud.fr
```

and then answer the following questions:

² Remember that this “packet number” is assigned by Wireshark for listing purposes only; it is NOT a packet number contained in any real packet header.

- Q16.** To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server?
- Q17.** Examine the DNS query message. How many questions does the query have? Does the query message contain any “answers”?
- Q18.** Examine the DNS response message. How many answers does the response have? What information is contained in the answers? How many additional resource records are returned? What additional information is included in these additional resource records?